



Artificial Intelligence Foundations (CIS4049-N)

Problem-solving III: Informed Search

Module Leader: Dr Alessandro Di Stefano

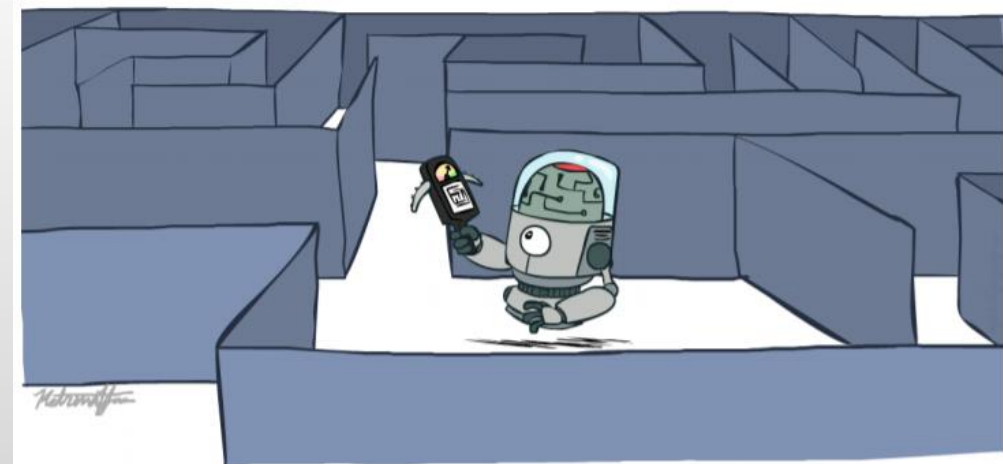
Email: A.DiStefano@tees.ac.uk

School of **Computing, Engineering & Digital Technologies**

tees.ac.uk

Today's Plan

- **Informed Search**
 - Heuristics
 - Greedy Search
 - A* Search
 - Admissible Heuristics
 - Designing Heuristics - Examples
- **Graph Search**
 - Consistent Heuristics



Wrap-up: Search

- **Search problem**

- **States** (configurations of the world)
- **Actions** and **costs**
- **Successor function** (world dynamics)
- **Start state** and **goal test**

- **Search tree**

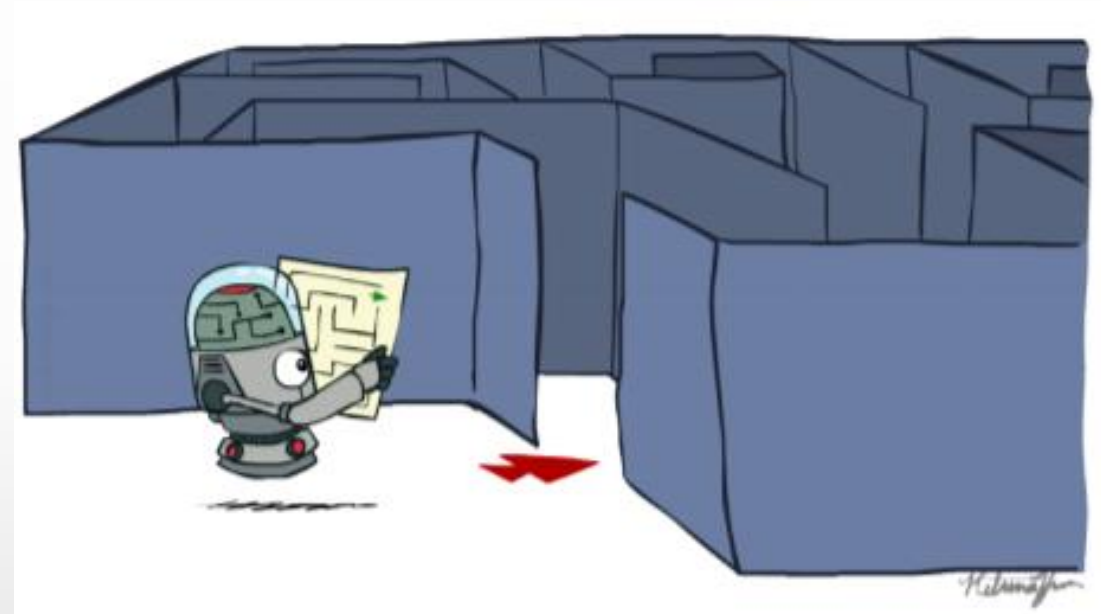
- **Nodes**: represent plans for reaching states
- **Plans** have **costs** (sum of action costs)

- **Search algorithm**

- Systematically **builds a search tree**
- **Orders the fringe** (unexplored nodes)

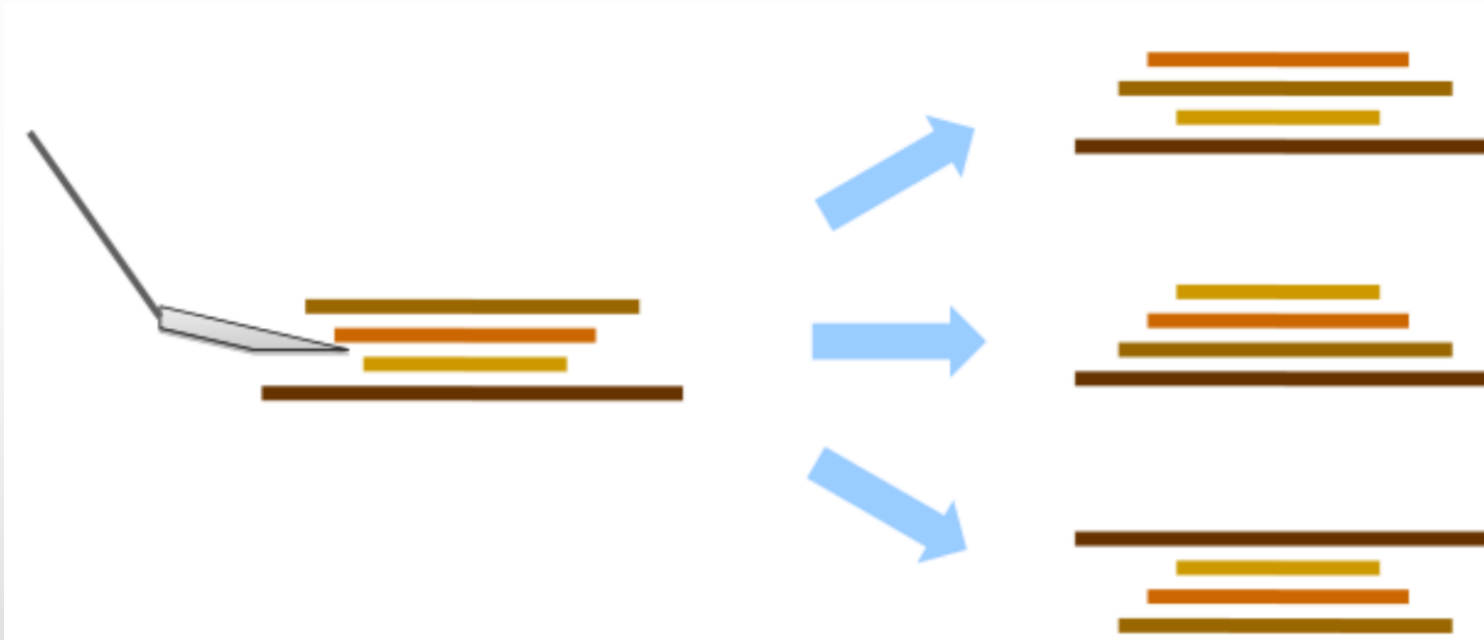
- **Complete** - finds a plan

- **Optimal** - finds least-cost plans



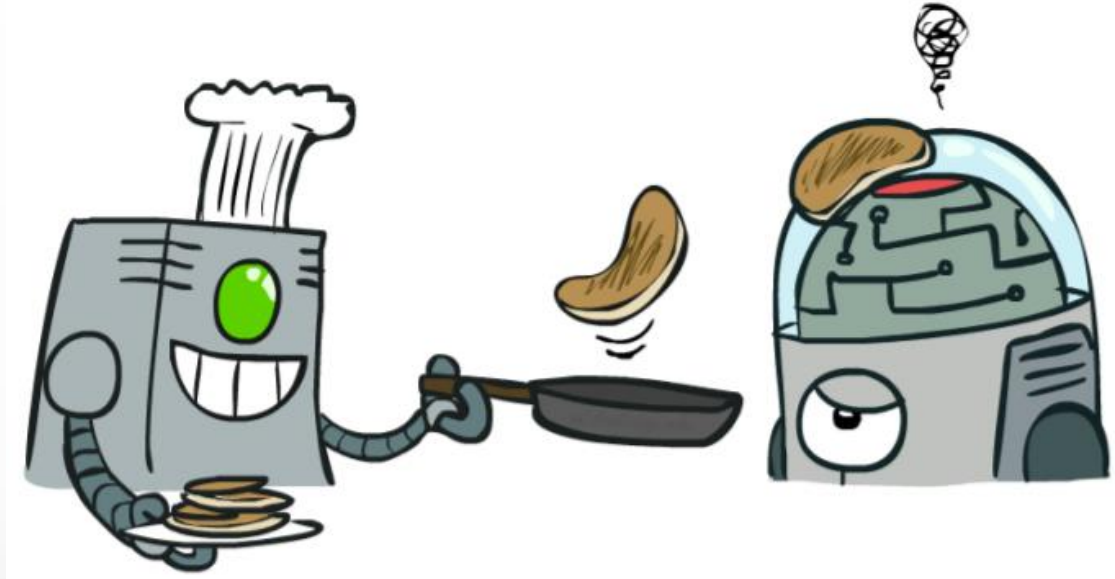
Example – Pancake Problem

Goal: Get the pancakes stacked with the biggest one at the bottom, and increasingly lower size (smallest pancake at the top).



Cost: Number of pancakes flipped

Example – Pancake Problem



BOUNDS FOR SORTING BY PREFIX REVERSAL

William H. GATES

Microsoft, Albuquerque, New Mexico

Christos H. PAPADIMITRIOU^{*†}

Department of Electrical Engineering, University of California, Berkeley, CA 94720, U.S.A.

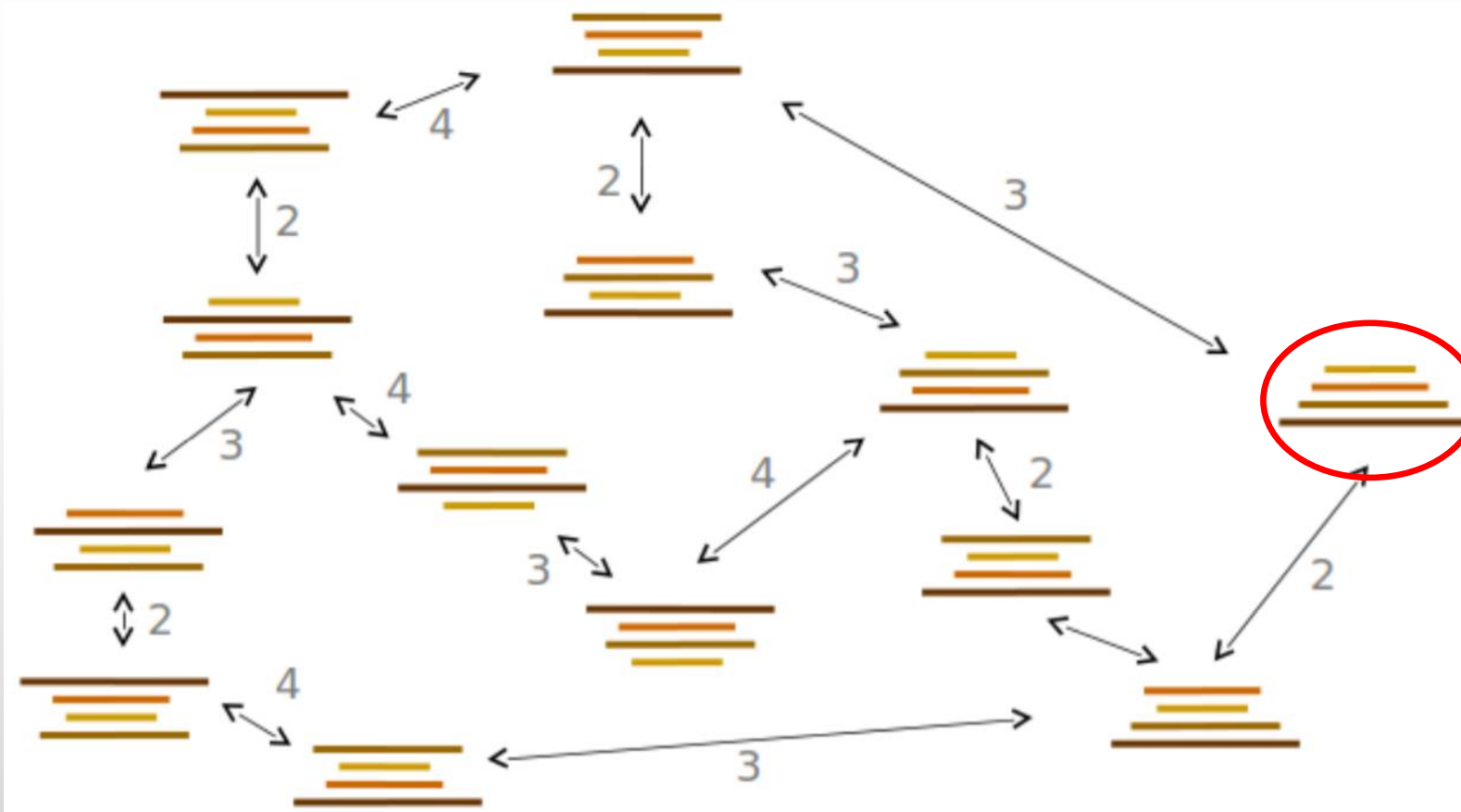
Received 18 January 1978

Revised 28 August 1978

For a permutation σ of the integers from 1 to n , let $f(\sigma)$ be the smallest number of prefix reversals that will transform σ to the identity permutation, and let $f(n)$ be the largest such $f(\sigma)$ for all σ in (the symmetric group) S_n . We show that $f(n) \leq (5n+5)/3$, and that $f(n) \geq 17n/16$ for n a multiple of 16. If, furthermore, each integer is required to participate in an even number of reversed prefixes, the corresponding function $g(n)$ is shown to obey $3n/2 - 1 \leq g(n) \leq 2n + 3$.

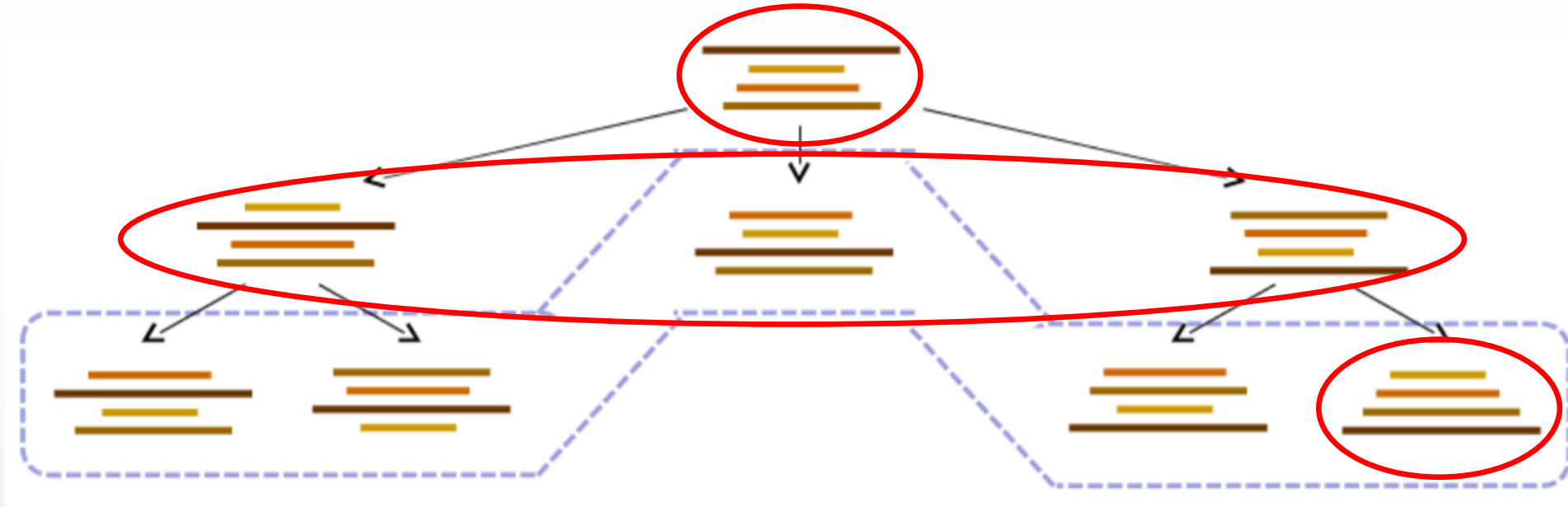
Example – Pancake problem

- **State space graph with costs as weights**



$$4 \times 3 \times 2 \times 1 = 24$$

Example – Pancake problem



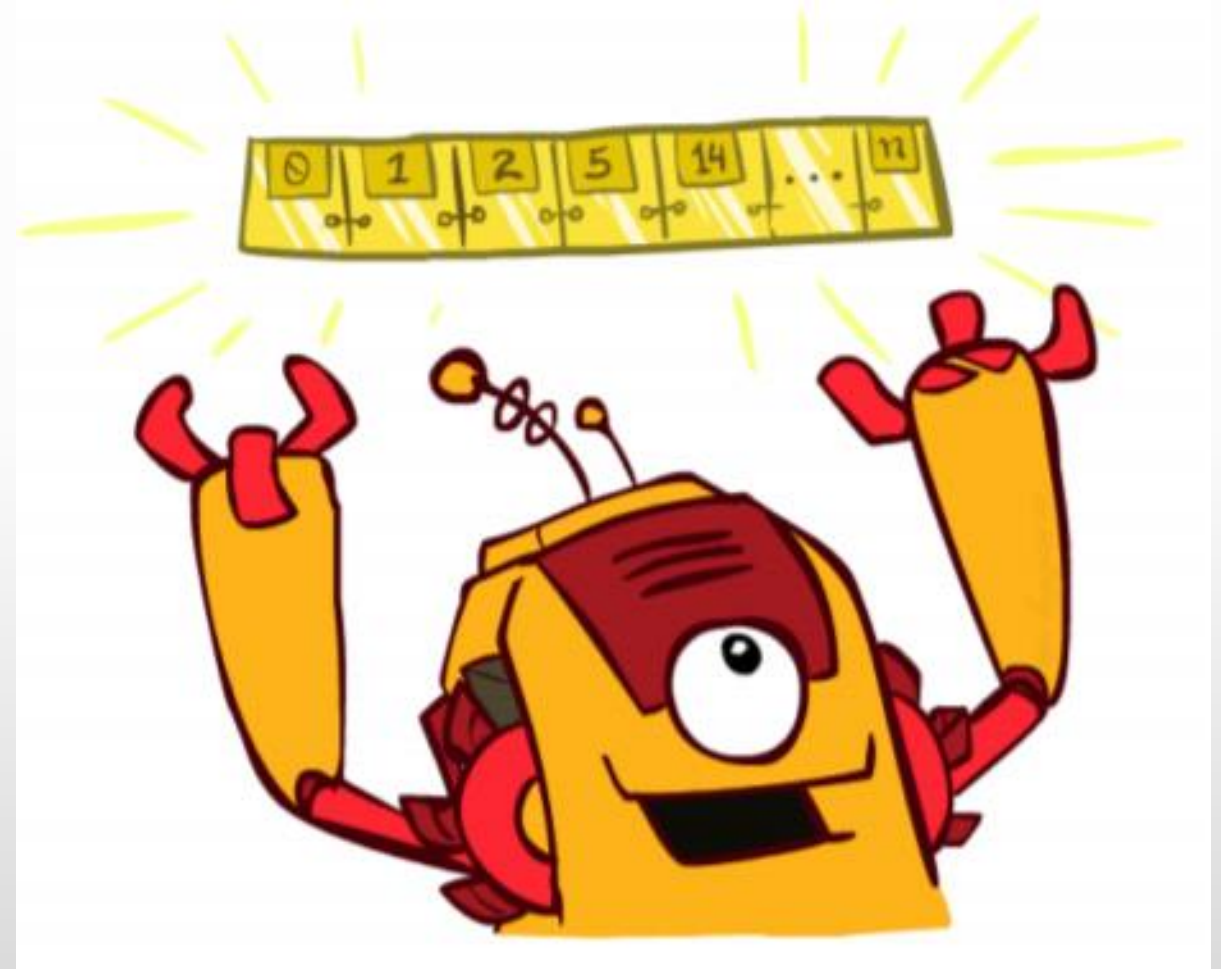
- **Action:** flip flop **two** -> Cost: 2
- **Action:** flip **all four** -> Cost: 4
- **Path** to reach **goal**: Flip four, flip three
-> Total cost: 7

According to a **strategy**

```
function TREE-SEARCH(problem) returns a solution, or failure
  initialize the frontier using the initial state of problem
  loop do
    if the frontier is empty then return failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state then return the corresponding solution
    expand the chosen node, adding the resulting nodes to the frontier
```

The one Queue

- All these search algorithms are the same except for fringe strategies
- Conceptually, all fringes are **priority queues** (i.e., collections of nodes with attached priorities)
- Practically, for DFS and BFS, you can avoid the **$\log(n)$** overhead from an actual priority queue, by using stacks and queues
- Can even code one implementation that takes a variable queuing object

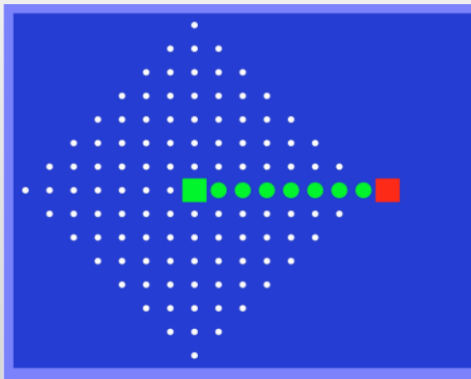
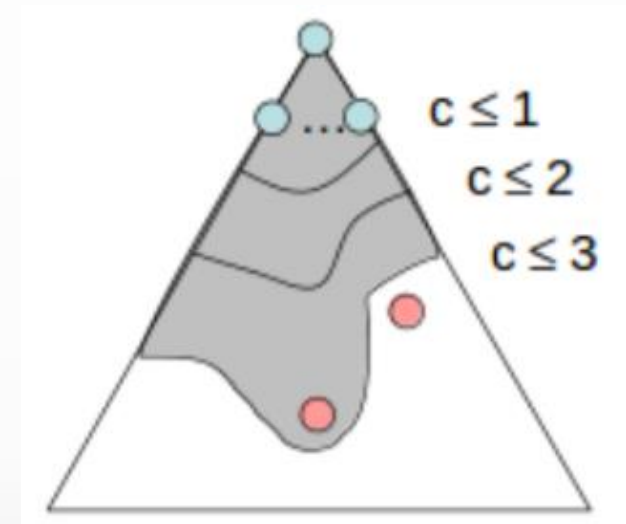


Uninformed Search

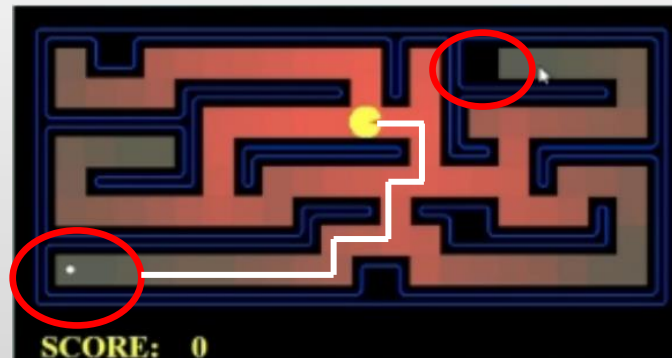


Uniform Cost Search – Issues

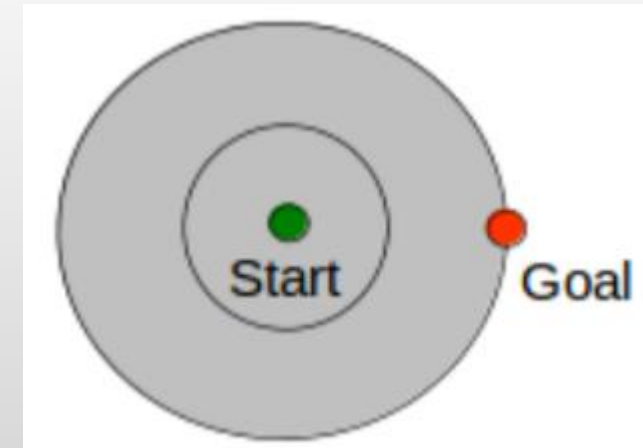
- **Strategy:** expand lowest path cost
- The **good**: **complete** and **optimal**!
- The **bad**:
 - Goes in **every “direction”**
 - **No information** about **goal location**



Goal
state



The only state
not expanded

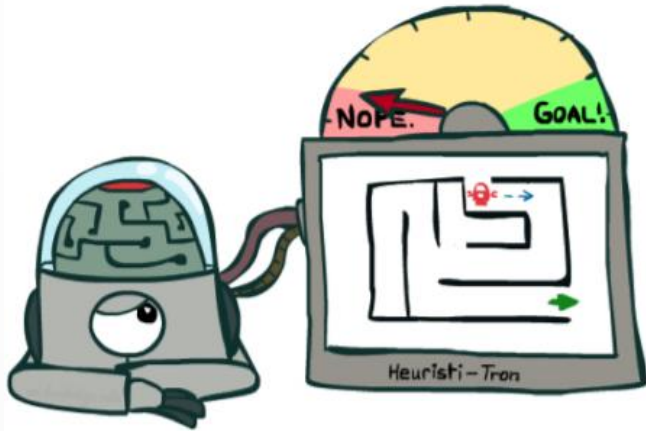




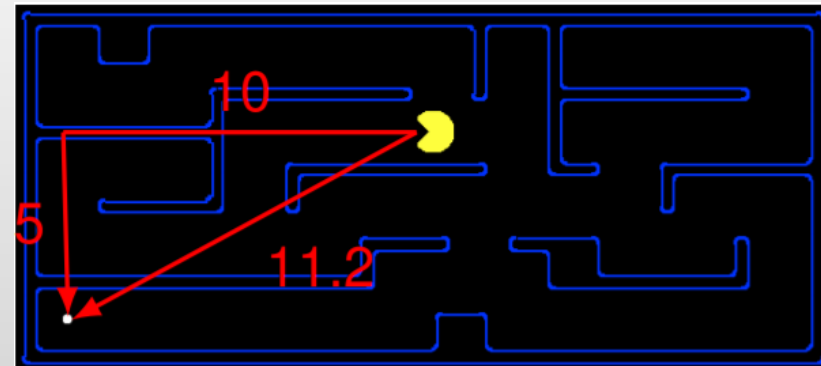
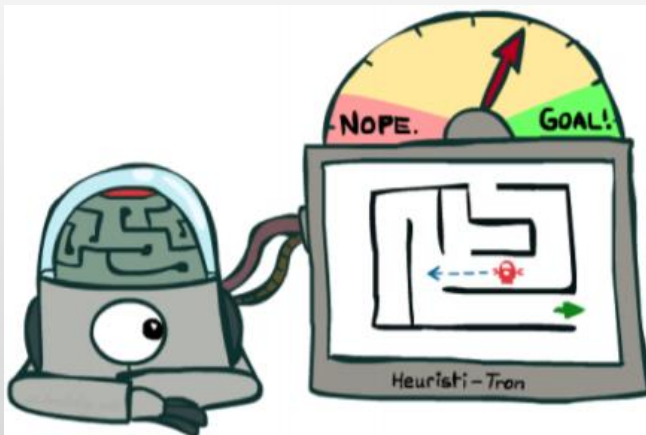
Informed Search

Heuristics

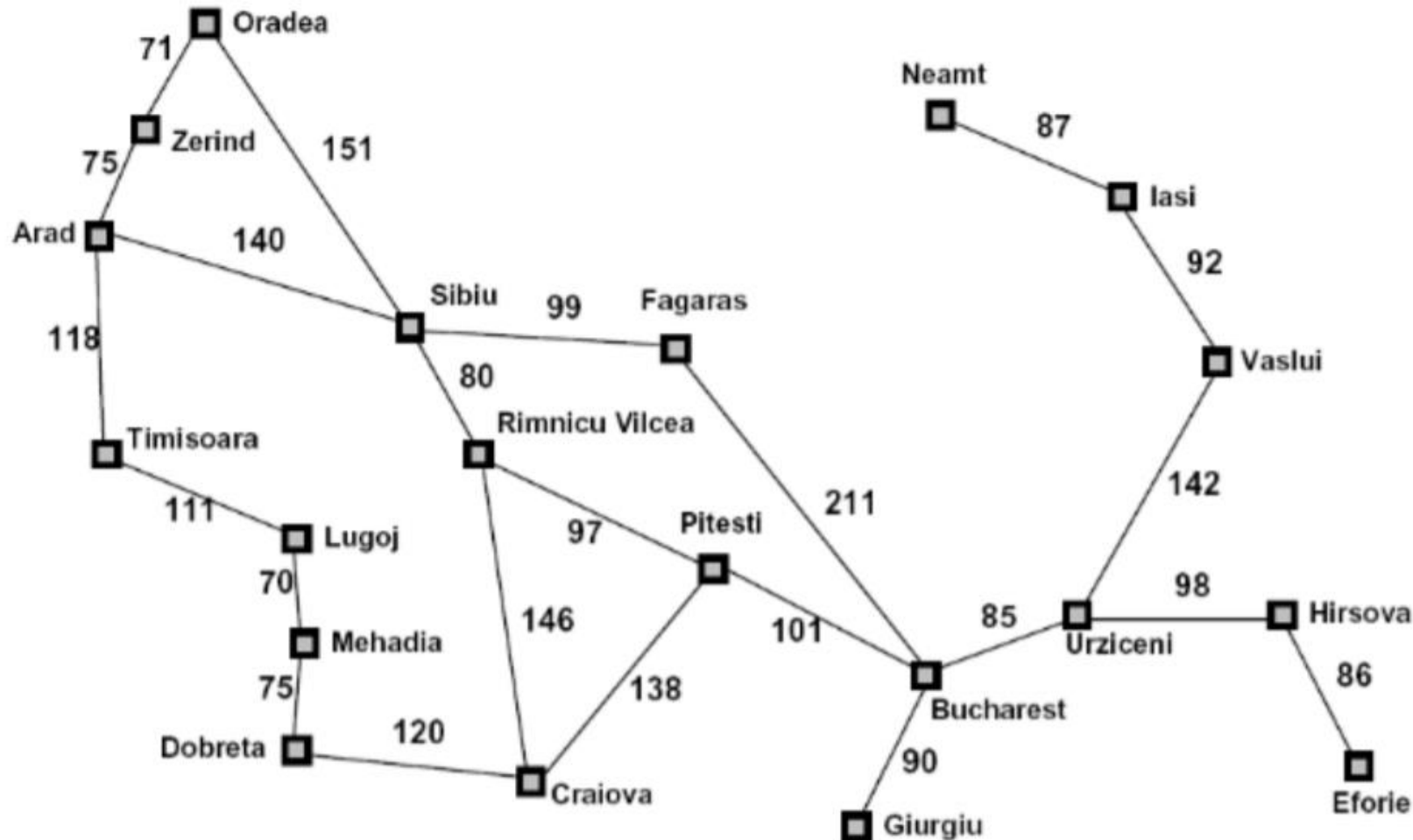
Search Heuristics



- A **heuristic** is:
 - A **function** that *estimates how close* a state is to a **goal**
 - Designed for a particular search problem
 - Examples:
 - Euclidean distance for pathing
 - Manhattan distance



Heuristic Function - Example



Straight-line distance
to Bucharest

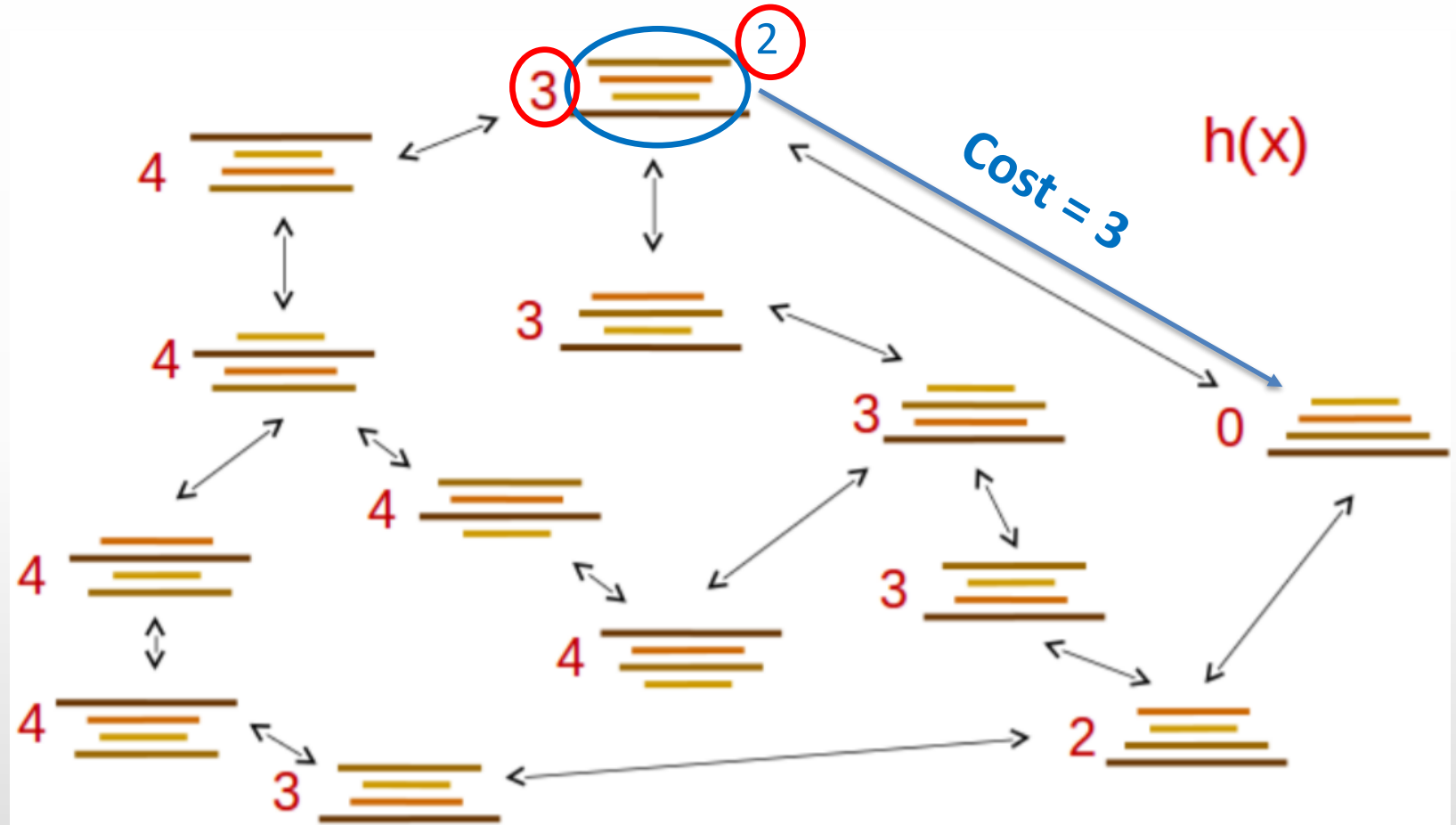
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

$h(x)$

Heuristic Function - Example

Heuristic - the ID number of the largest pancake that is still out of place

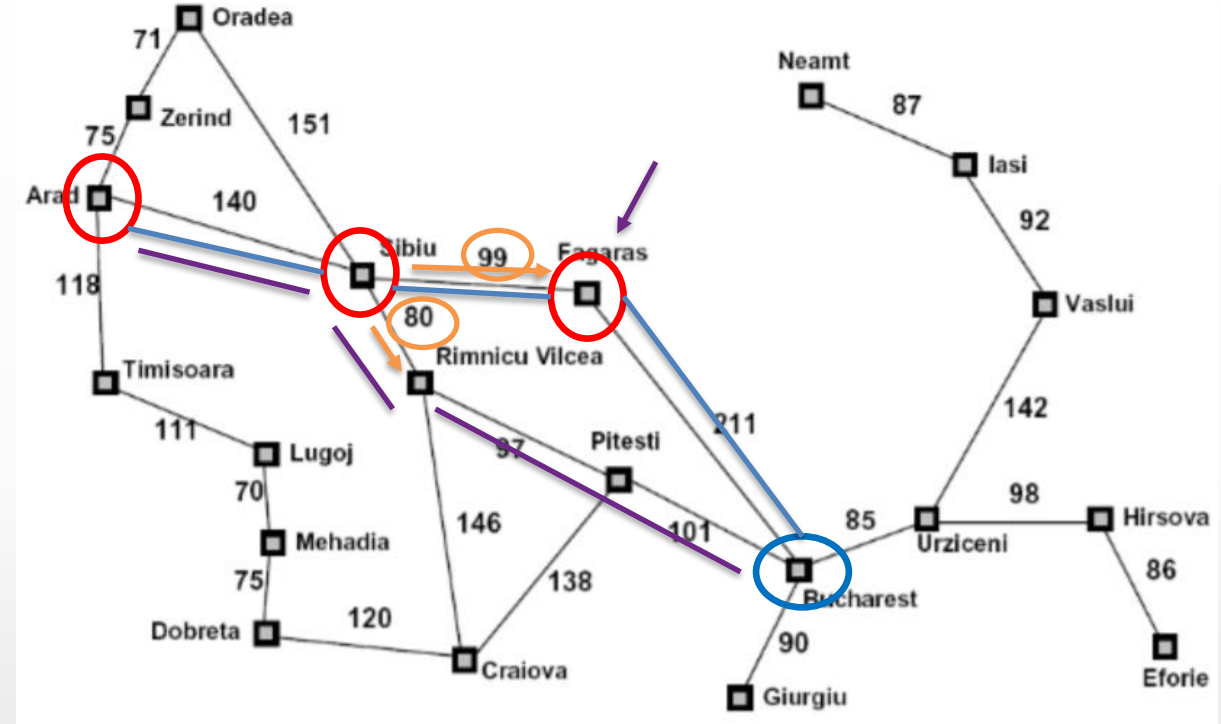
Another Heuristic - number of pancakes still out of place





Greedy Search

Greedy Search

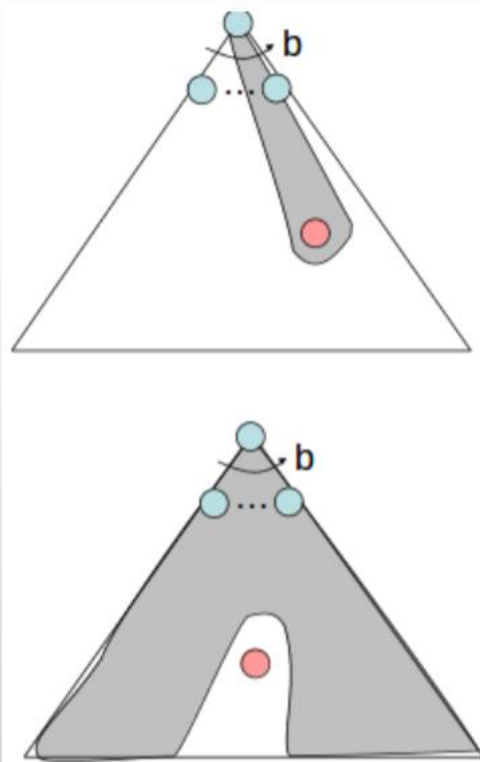


What can go wrong?

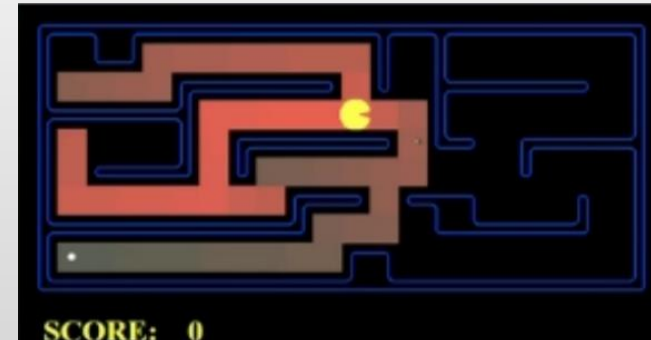
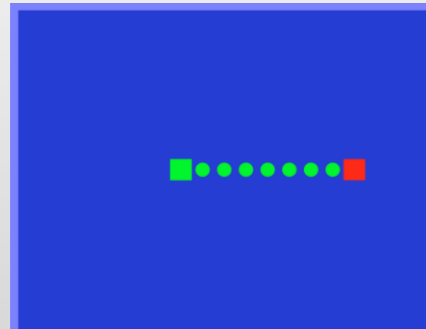
- **Greedy:** $140 + 99 + 211 = 450$
- **Better:** $140 + 80 + 97 + 101 = 418$

Greedy Search

- Strategy: expand a node that (you think) is closest to a goal state
- **Heuristic:** estimate of distance to nearest goal for each state



- “Wildest” dream: Perfect heuristic
- A common case: Best-first takes you straight to the (wrong) goal
- Worst-case: Like a badly-guided DFS



Greedy



A* Search

A* Search



UCS



Greedy

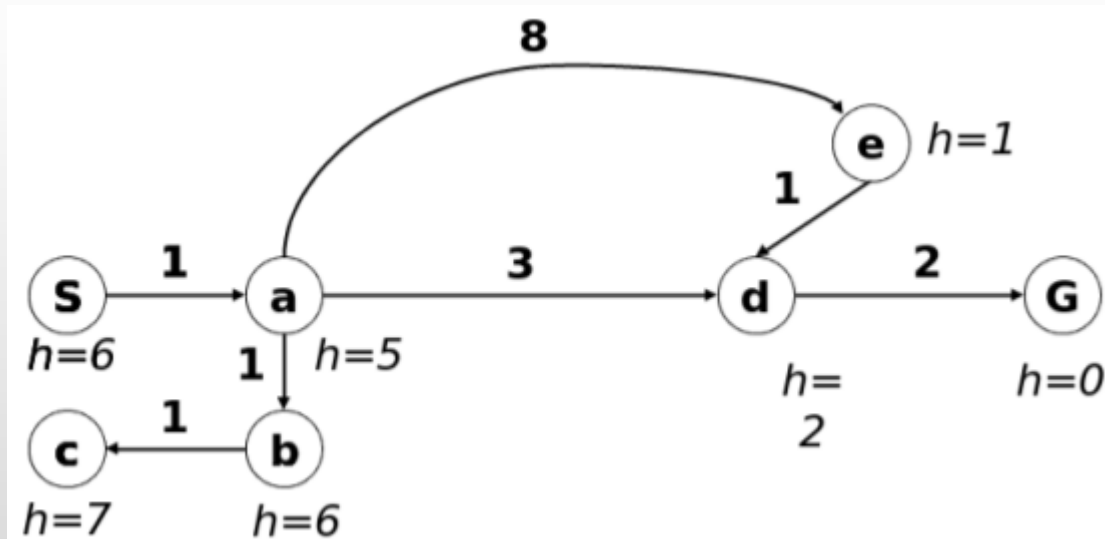


A*

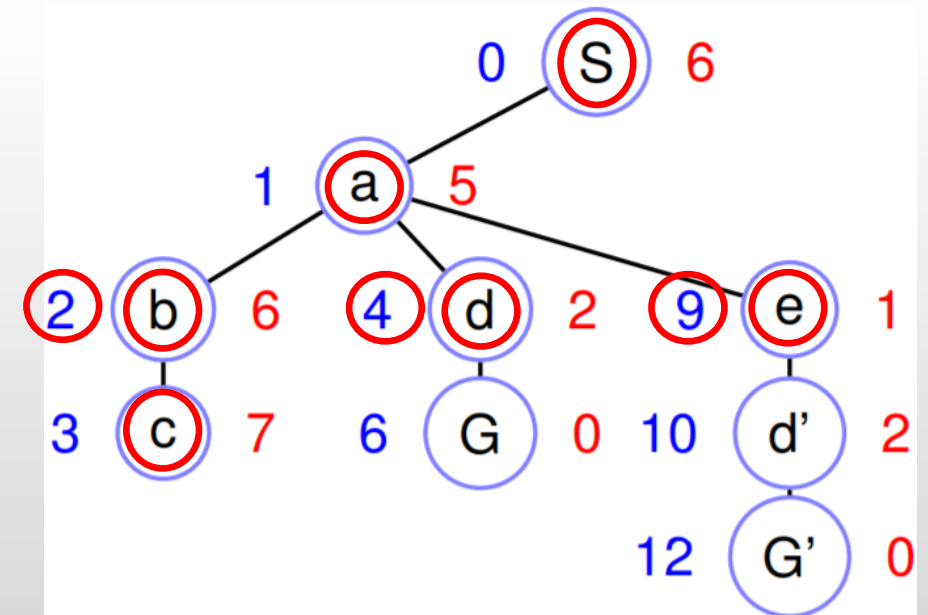
Combining UCS and Greedy

- **Uniform-cost** orders by path cost, or backward cost: $g(n)$
- **Greedy** orders by goal proximity, or forward cost: $h(n)$
- **A* Search** orders by the sum: $f(n) = g(n) + h(n)$

Space state graph

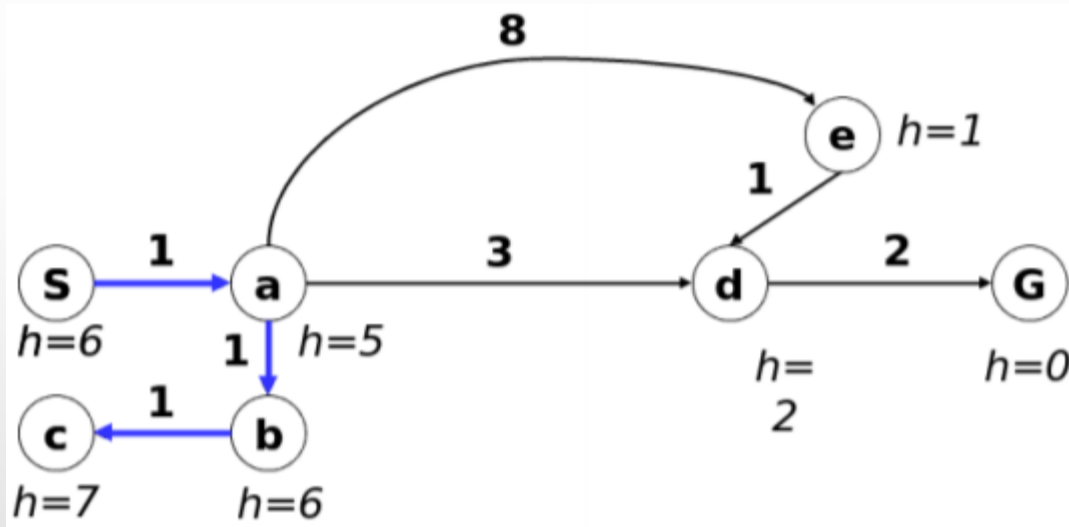


Search tree

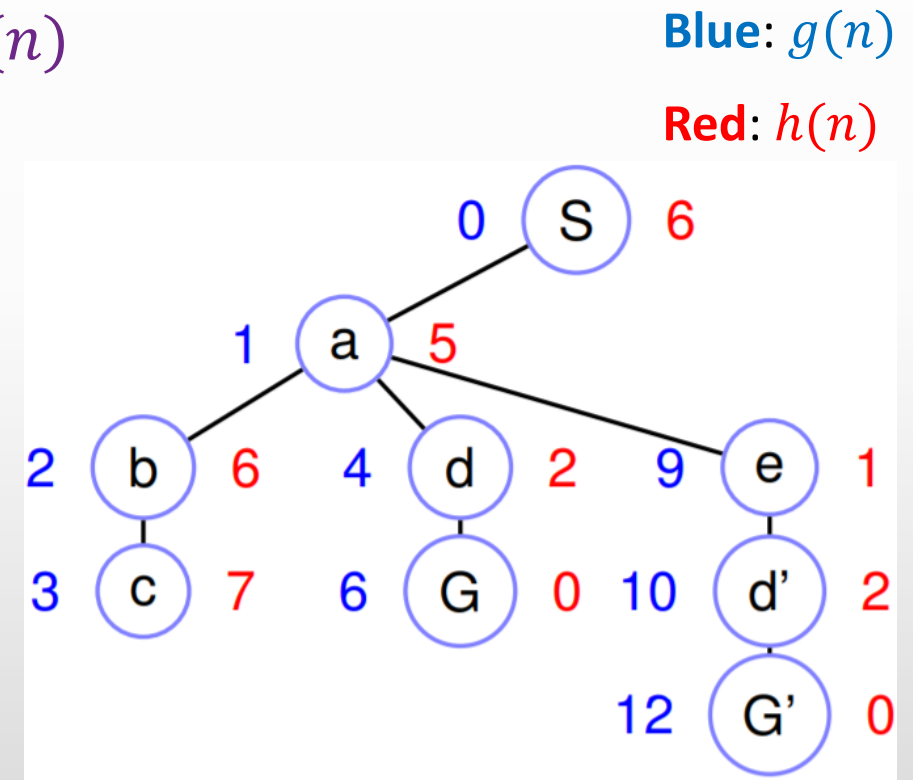


Combining UCS and Greedy

- **Uniform-cost** orders by path cost, or backward cost: $g(n)$
- **Greedy** orders by goal proximity, or forward cost: $h(n)$
- **A* Search** orders by the sum: $f(n) = g(n) + h(n)$

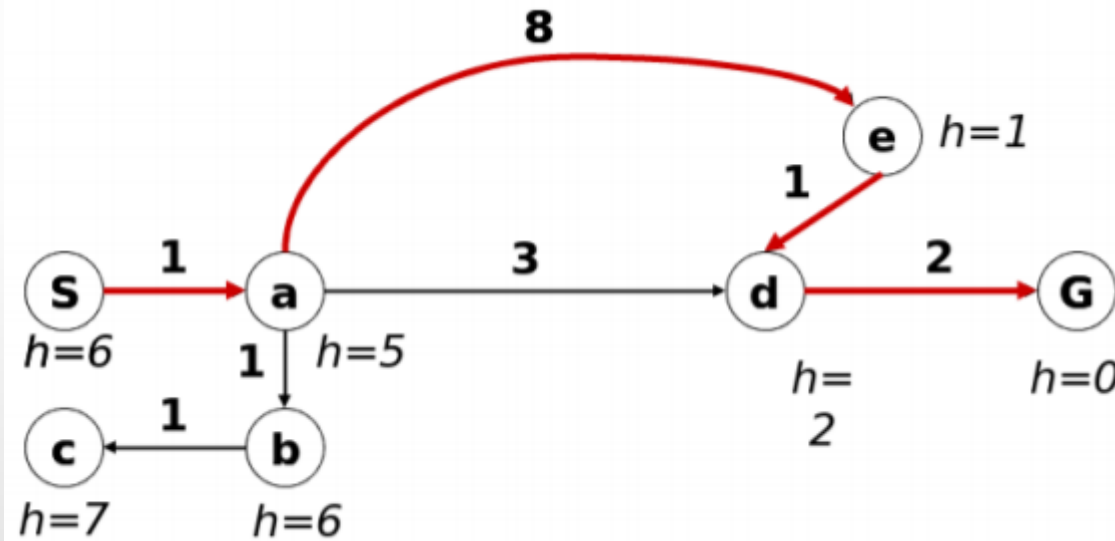


UCS



Combining UCS and Greedy

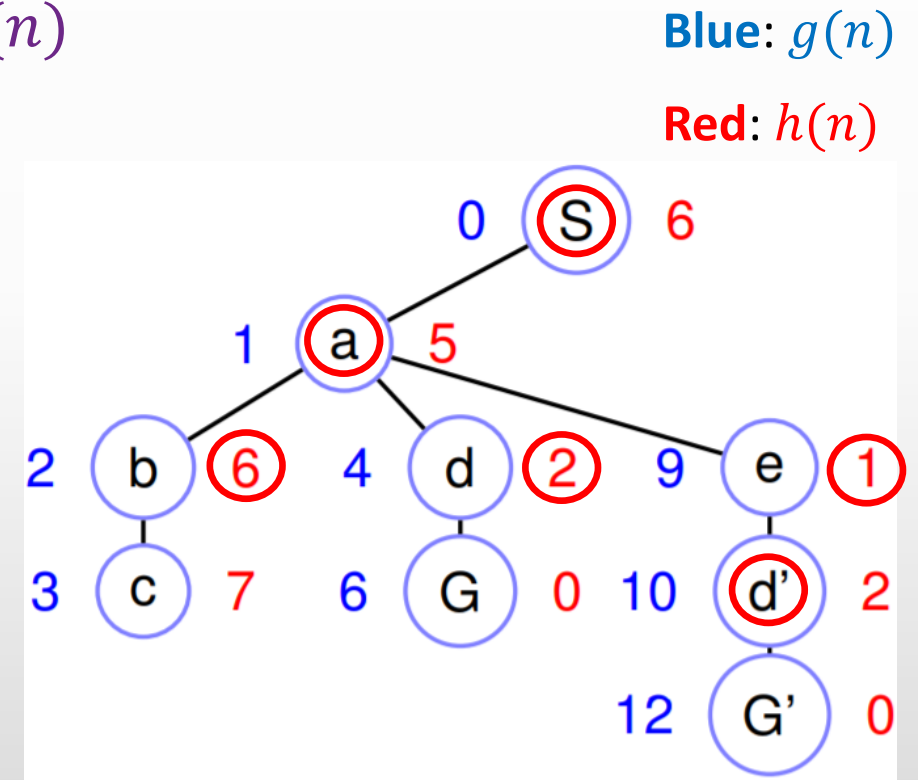
- **Uniform-cost** orders by path cost, or backward cost: $g(n)$
- **Greedy** orders by goal proximity, or forward cost: $h(n)$
- A* Search orders by the sum: $f(n) = g(n) + h(n)$



UCS

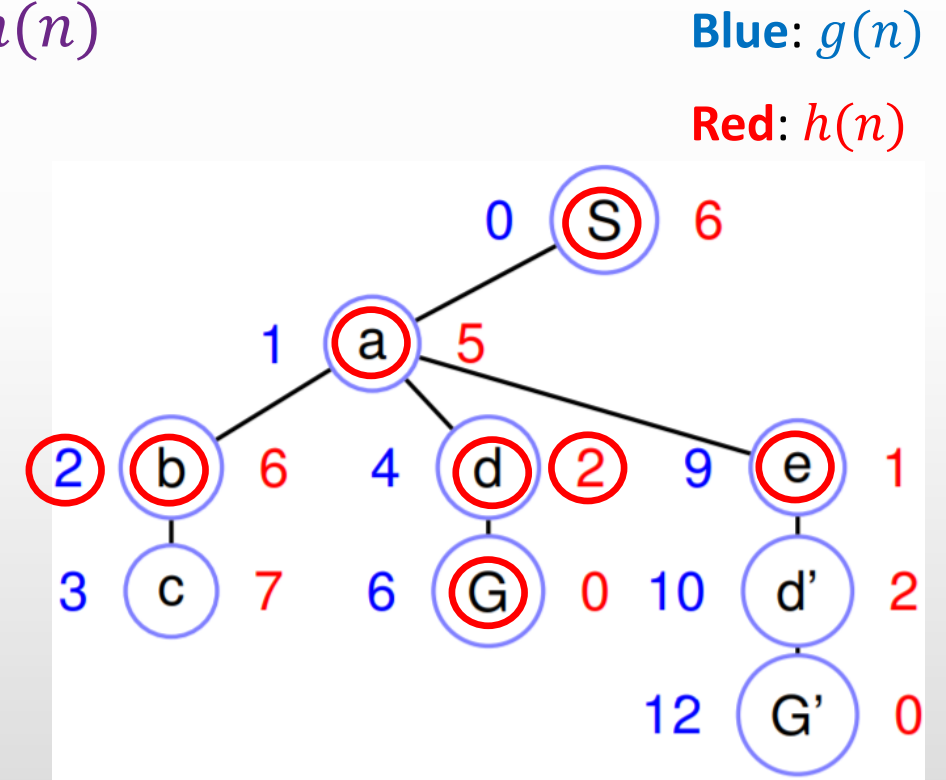
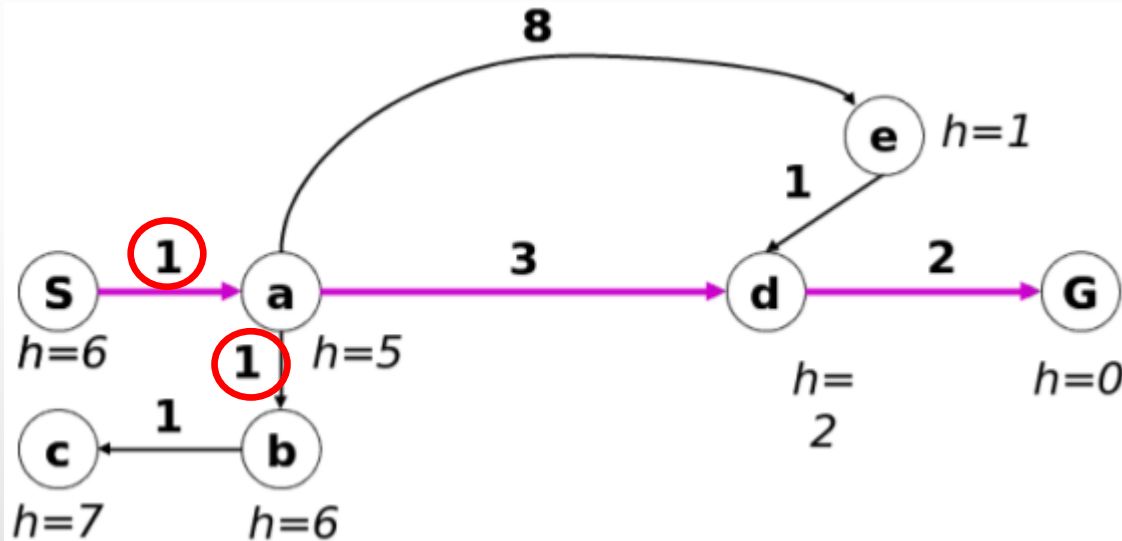


Greedy



Combining UCS and Greedy

- **Uniform-cost** orders by path cost, or backward cost: $g(n)$
- **Greedy** orders by goal proximity, or forward cost: $h(n)$
- **A* Search** orders by the sum: $f(n) = g(n) + h(n)$



UCS



Greedy



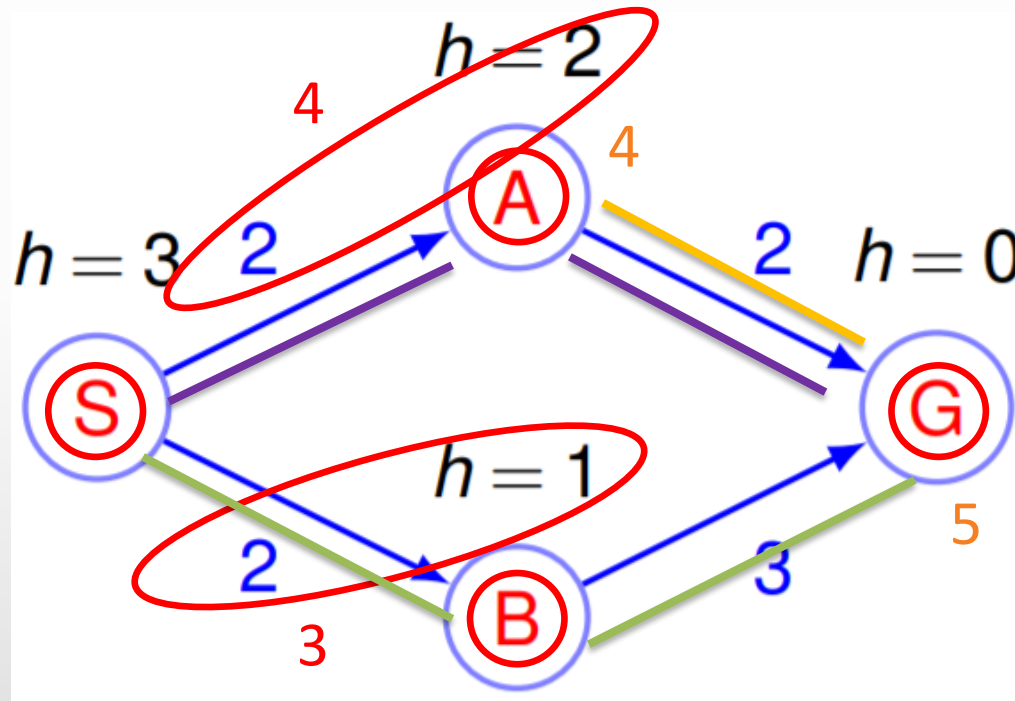
A*

When Should A* Terminate?



A*

- Should we stop when we enqueue a **goal**?



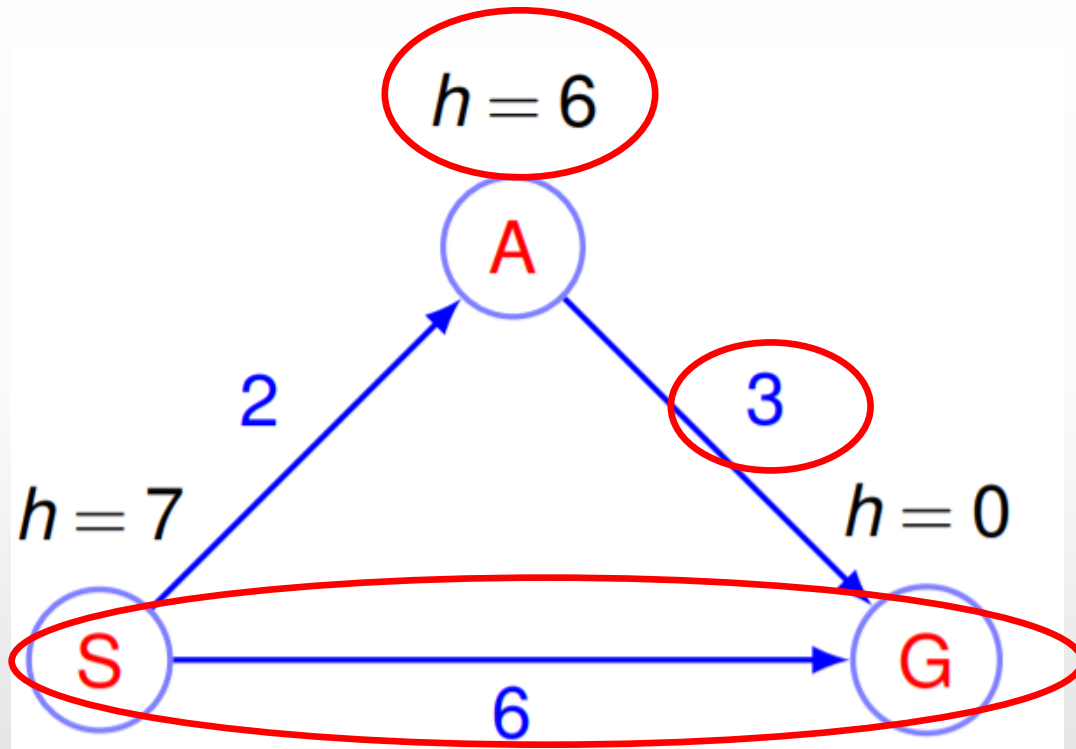
- G in queue when B expanded
- No: only stop when we dequeue a goal**

S->A->G 4
S->B->G 5

Is A* Optimal?

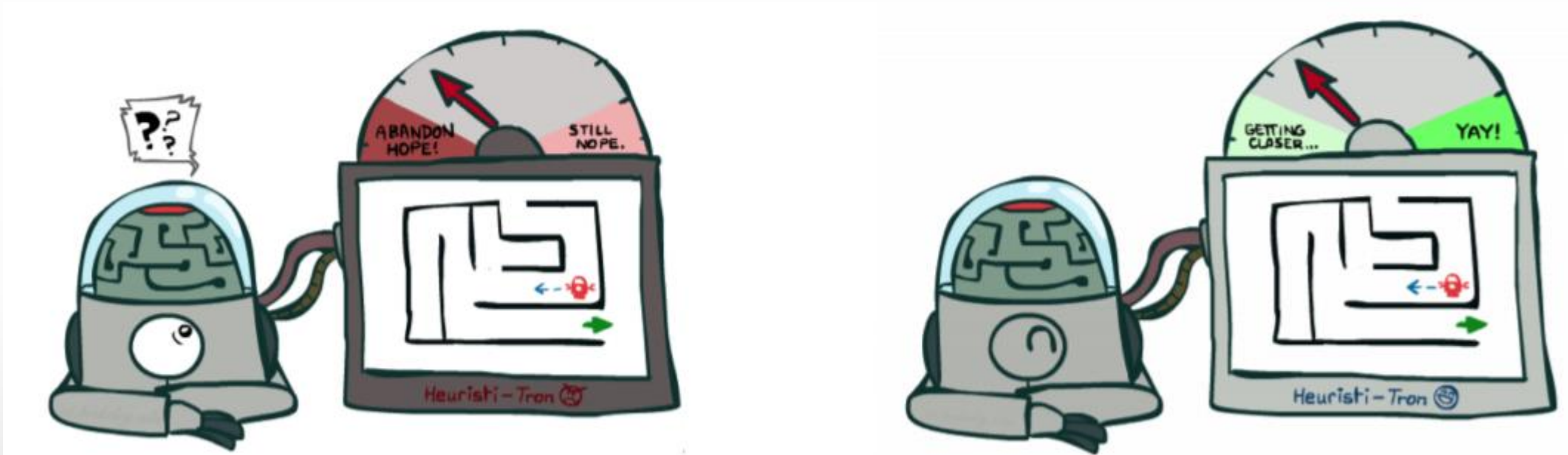


A*



- What goes wrong?
- Actual bad goal cost < estimated good goal cost (i.e., $h = 6$ is too pessimistic)
- We need **estimates to be less than actual costs!** - Lower bounds for actual costs

Admissible Heuristics - Idea



Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe (not expanding them as the heuristic is holding them back)

vs

Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

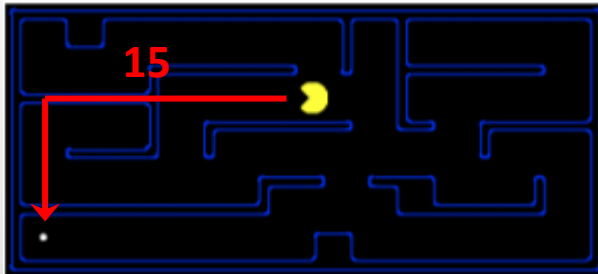
Admissible Heuristics

- A heuristic h is **admissible (optimistic)** if: $0 \leq h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to a nearest goal

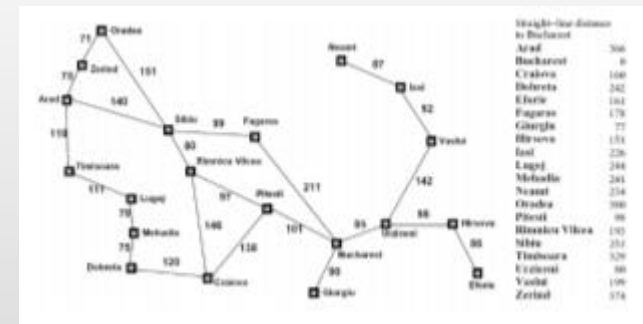
- Examples:



- Optimize:** number of flips
- Number (ID) of the largest out of place pancake – Admissible? Yes!!**
Since at the very least, you need to still go as deep as that one to rearrange pancakes, so flipping at least that many pancakes to get to the goal



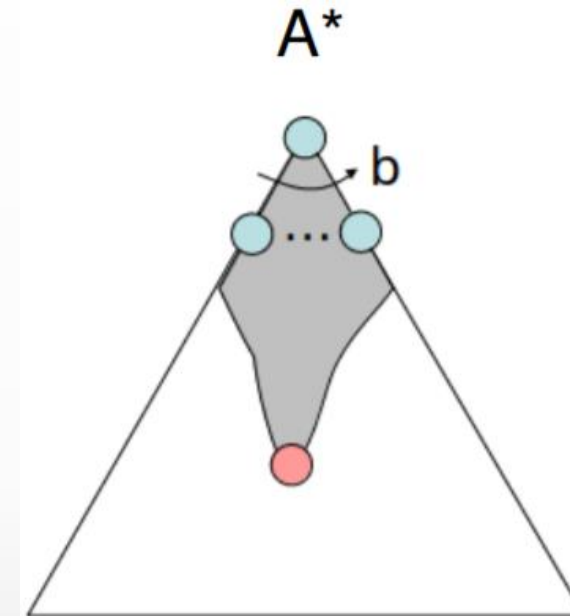
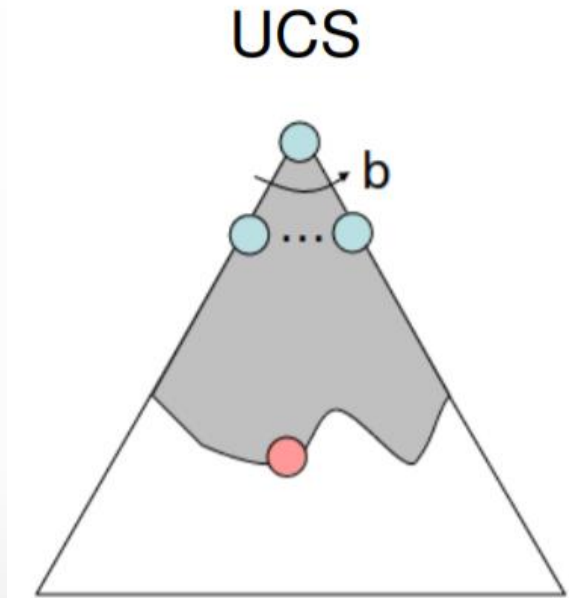
- Manhattan distance**
- Admissible? Yes!!**



- Euclidean distance**
- Admissible? Yes**

Note: Coming up with admissible heuristics is most of what's involved in using A* in practice

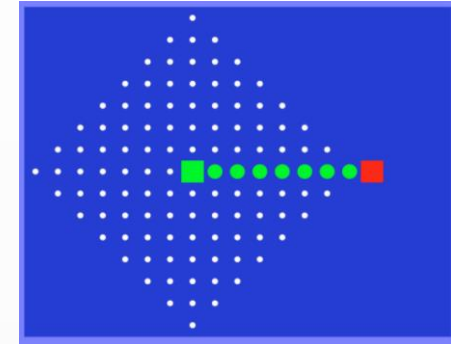
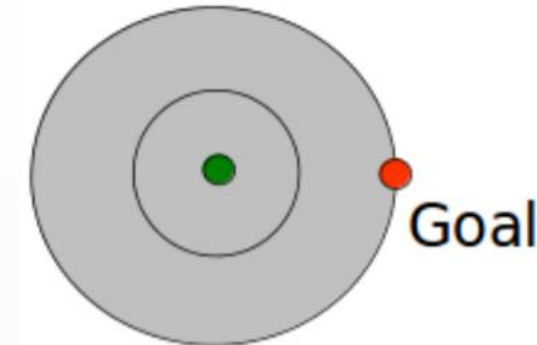
Properties of A*



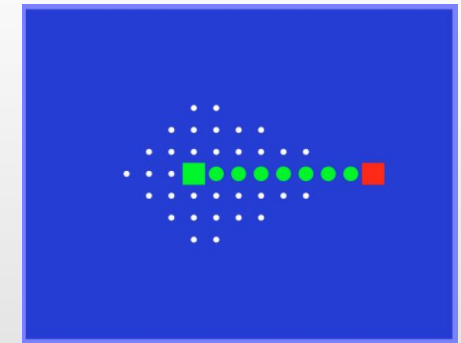
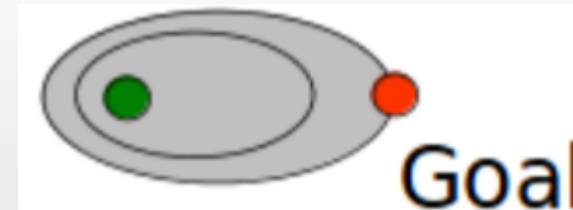
- **UCS** goes **equally fast** in terms of the **expansion in all directions**
- **A*** will **zone in** toward the goal if you have a reasonably **good heuristic**

UCS vs A* Contours

- **Uniform-cost** expands equally in all “directions”



- **A*** expands mainly **toward the goal**, but does hedge its bets to **ensure optimality**



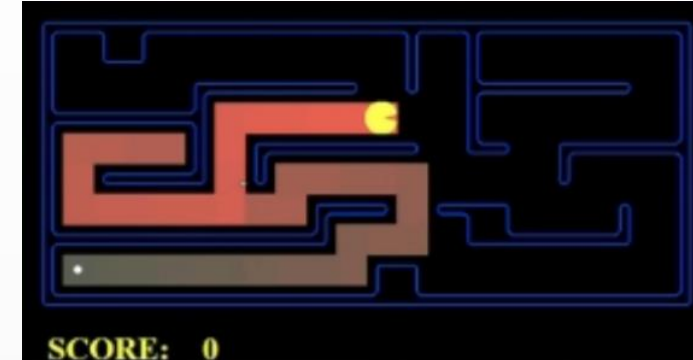
Heuristic - take the straight line distance to the goal and then take half of that value - things that are closer to the goal get favoured, but maybe not favoured as much as you might want.

Comparison – UCS vs Greedy vs A*

Uniform-cost (UCS)



Greedy



A*



- What will be expanded in A*?
- Any state that has cost plus heuristic lower than cheapest path to the goal will be expanded

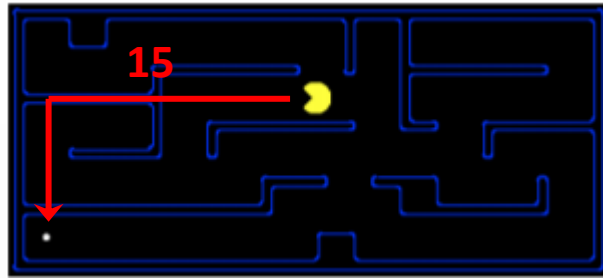
A* Applications

- Video games
- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Machine translation
- Speech recognition
- ...

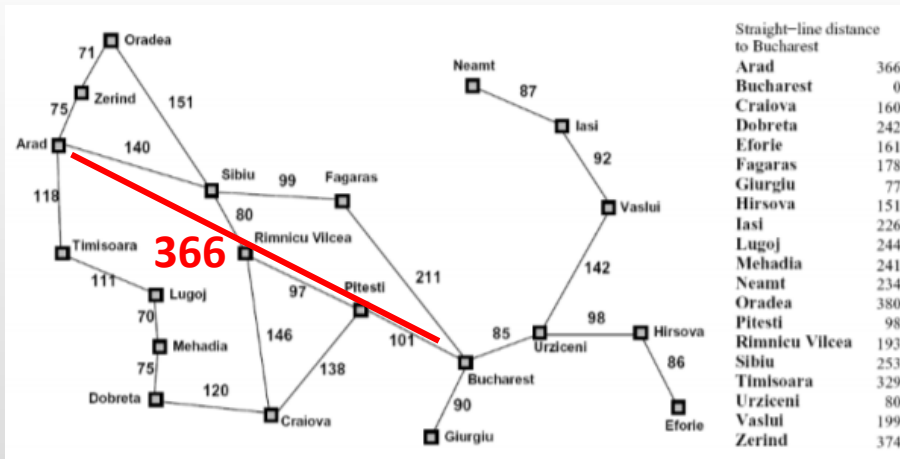


Designing Heuristics - Examples

Creating Admissible Heuristics



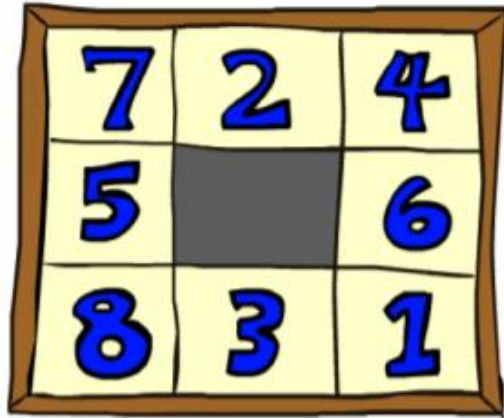
Manhattan Distance



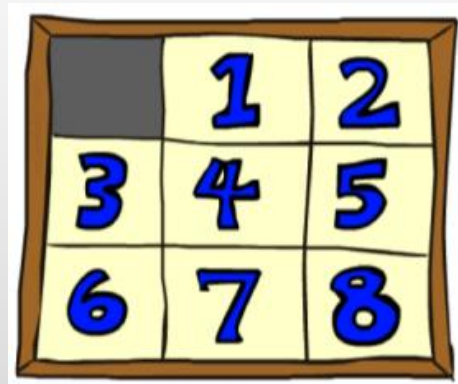
Straight Line Distance

- Most of the work in solving hard search problems optimally is in coming up with admissible heuristics
- Often, admissible heuristics are solutions to **relaxed** problems, where **new actions** are **available**
- **Inadmissible heuristics** are often useful too
 - Lose optimality, but there might be informative
 - Your solution is faster but not the right solution, not the optimal one
 - You might sometimes just want to find a solution fast

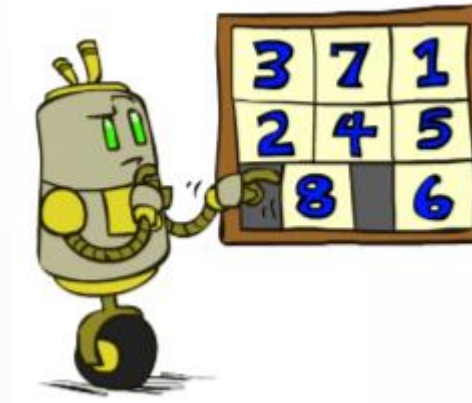
8-Puzzle



Start state



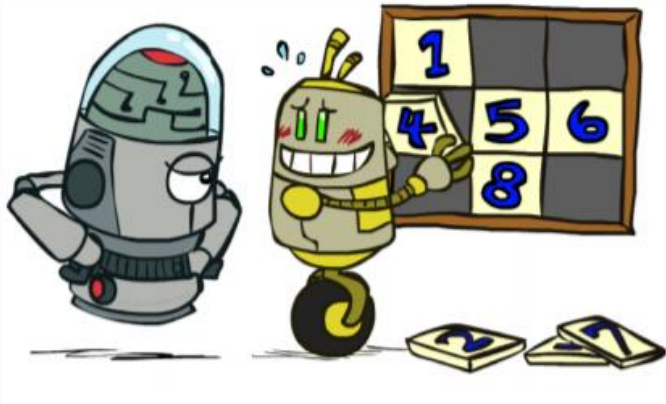
Goal state



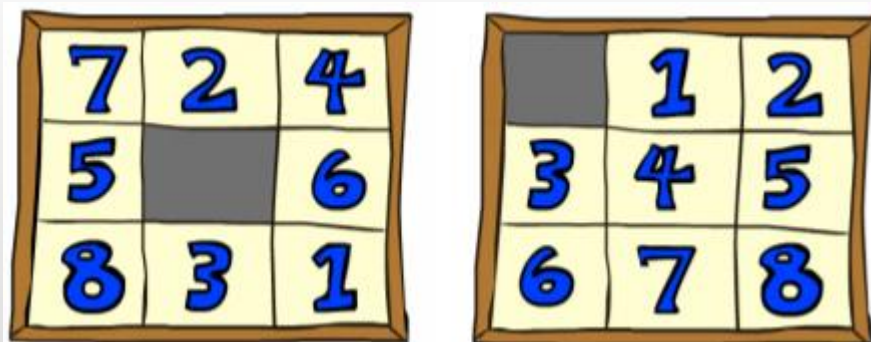
Actions

- What are the **states**? How many states? All the possible configurations of the board – 9! states
- What are the **actions**? You can move a tile onto the empty spot (move the empty spot around)
- How many **successors** from the **start state**? 4 tiles you can move into that, so 4 successors.
- What should the **costs** be? Designer choice, e.g., cost = 1

8-Puzzle - I



- **Heuristic:** number of tiles misplaced
- Why is it **admissible**? $h(start) = 8$
- This is a *relaxed-problem* heuristic
- **Average nodes expanded** when the **optimal path** has:



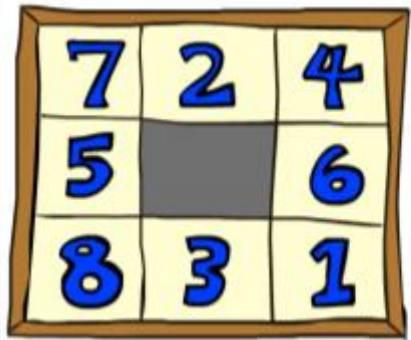
Start state

Goal state

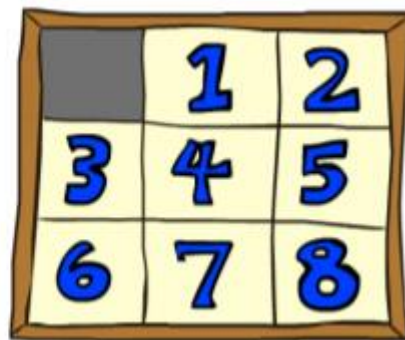
	4 Steps	8 Steps	12 Steps
UCS	112	6300	$3.6 \cdot 10^6$
TILES	13	39	227

Statistics from Andrew Moore

8-Puzzle - II



Start state



Goal state

- **Easier 8-puzzle:** tile could slide any direction at any time, ignoring other tiles
- **Total *Manhattan* distance**
- Why is it **admissible**?
 - $h(start) = 3 + 1 + 2 + \dots = 18$
- **Average nodes expanded** when the **optimal path** has:

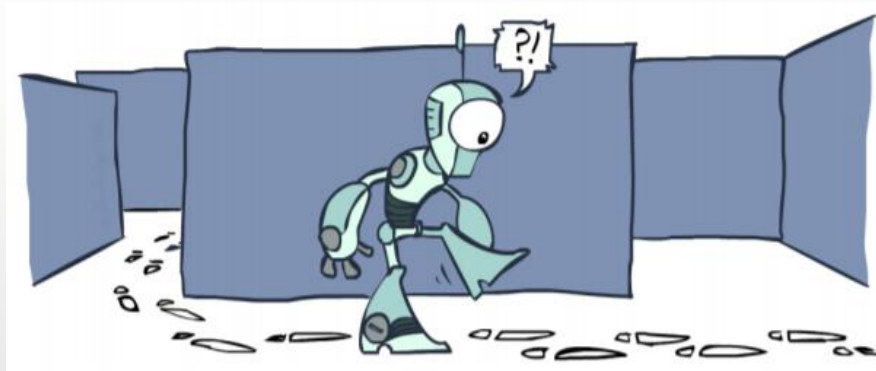
	4 Steps	8 Steps	12 Steps
TILES	13	39	227
MANHATTAN	12	25	73

8-Puzzle – III

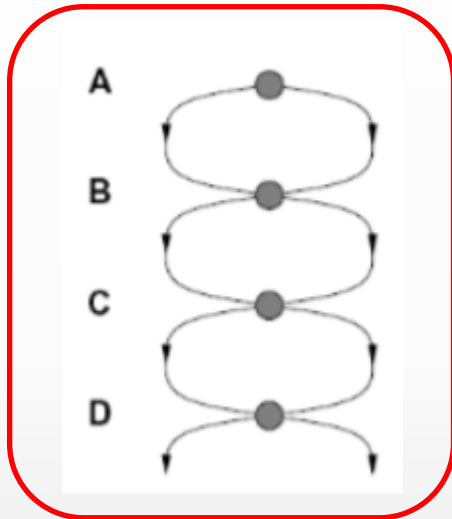
- How about using the **actual cost** as a **heuristic**?
- Would it be **admissible**? **Yes, actual cost = actual cost**
- Would we save on nodes expanded? **Yes, a lot!!**
- What's wrong with it? **We don't know the actual cost!!!**
- With **A***: a **trade-off** between **quality of estimate** and **work per node** (computational cost required to compute your heuristic and the number of nodes expanded)
 - As **heuristics** get **closer** to the **true cost**, expand **fewer nodes**, but **more work per node** to compute the heuristic itself



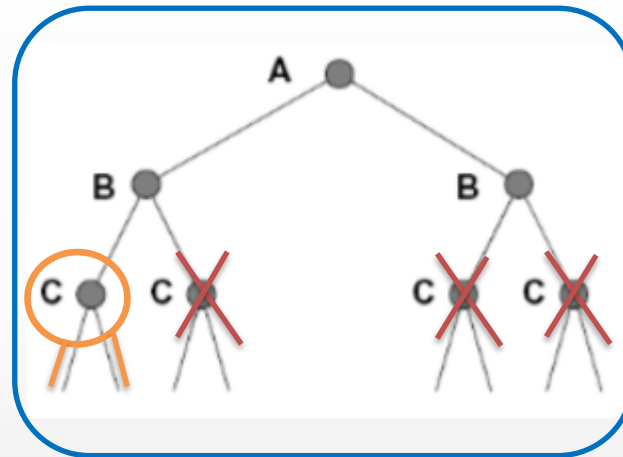
Graph Search



Tree Search: Extra Work!!



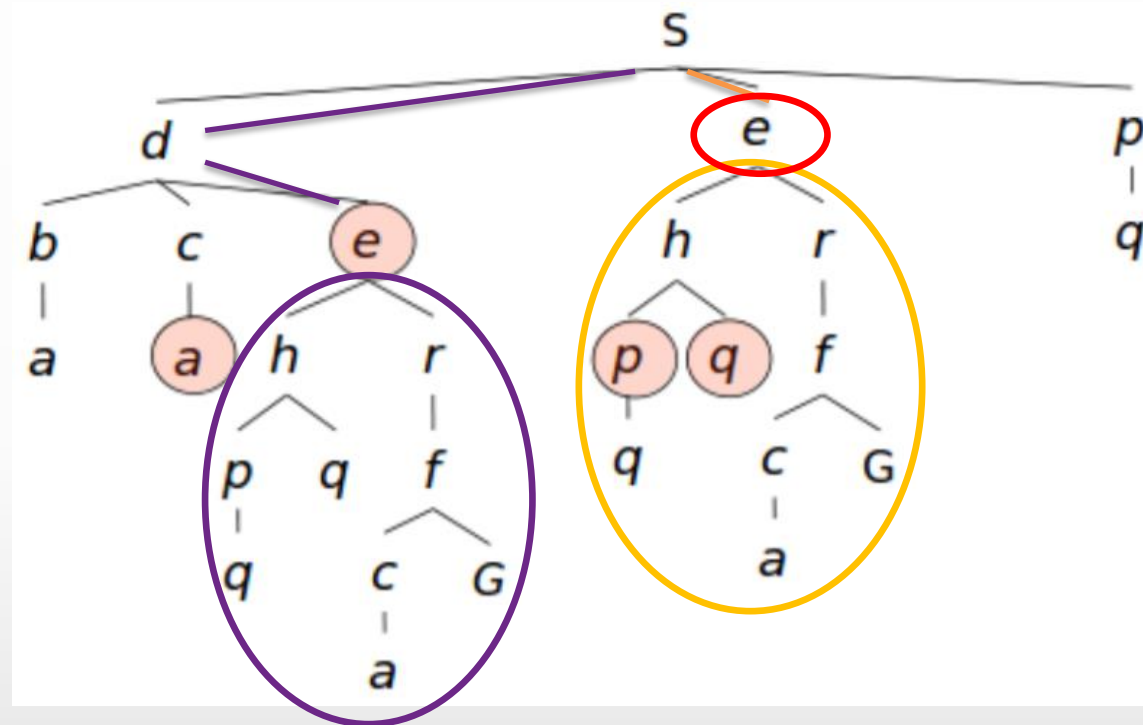
State Graph



Search Tree

Failure to detect repeated states can cause **exponentially** more work!

Graph Search



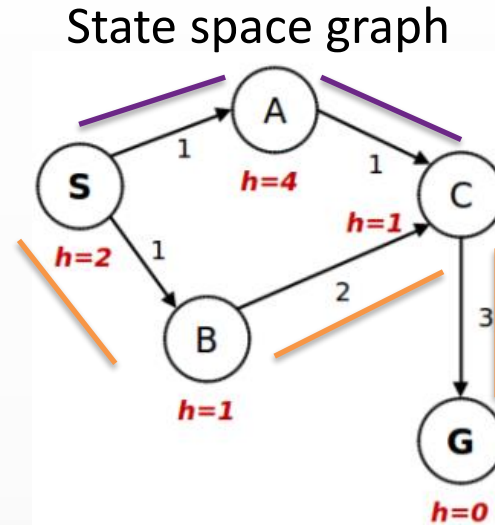
In BFS, for example, we shouldn't bother expanding the circled nodes (why?)

Graph Search

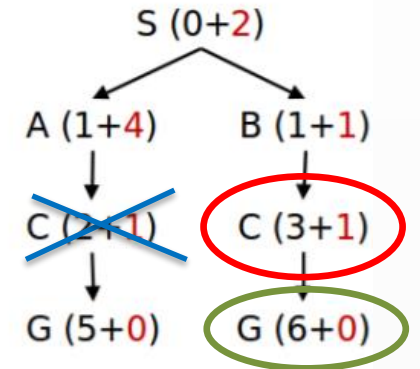
- **Idea:** never **expand** a state twice!!
- **How to implement:**
 - **Tree search** + set of **expanded states** (“closed set”)
 - Expand the search tree node-by-node, but...
 - Before expanding a node,
 - check if state was never been expanded before
 - If yes skip it, else add to closed set and expand
- **Important:** store the closed set as a **set**, not a **list** - time it takes to traverse the list and find whether you're in there or not is much worse than when it's stored as a set
- Can graph search wreck **completeness**? Why/why not?
- How about **optimality**?

A* Graph Search Gone Wrong?

- Is $h(\cdot)$ **admissible**? Yes
- Will exploring w.r.t. $h(B) + g(n)$ be **optimal**?
- Expand **S**
 - **A** and **B** in fringe!
 - Expands **B**, since $h(B) + g(B) = 2 < 5 = h(A) + g(A)$
 - **C** in fringe with key, $3 + h(C) = 4$
 - **G** in fringe with key, 6
- Could have been there in **5**

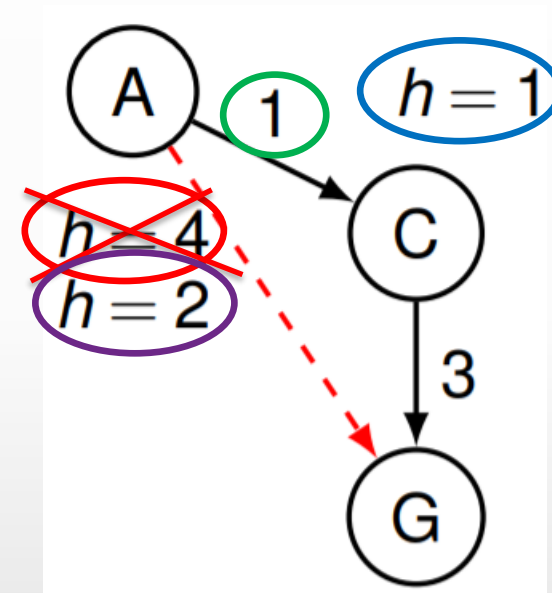


Search Tree



Consistency of Heuristics

- **Main idea:** estimated heuristic costs $\leq$ actual costs
- **Admissibility:** $h(x) \leq \text{cost to goal}$
- **Consistency:** $h(x) - h(y) \leq \text{cost}(x, y)$
 - heuristic “arc” cost $\leq$ actual arc cost
- Consistent $\Rightarrow$ admissible? Yes? No?
- Consistent: f value along a path never decreases
- Admissible:
 - $f(C) = h(C) + g(C) = 1 + 3 = 4$
 - $f(A) = g(A) + h(A) = 1 + 2 = 3$
- Consistent: $f(A) = 3 \leq 4 = f(C)$



$$h(A) \leq \text{cost}(x, y) + h(C)$$

$$f(A) = g(A) + h(A) \leq g(A) + \text{cost}(x, y) + h(C) = f(C)$$

Optimality

- Tree search:
 - A* is **optimal** if heuristic is **admissible**
 - UCS is a special case ($h = 0$)
- Graph search:
 - A* is **optimal** if heuristic is **consistent**
 - UCS optimal ($h = 0$ is consistent)
- **Consistency** implies **admissibility**
- In general, most natural admissible **heuristics** tend to be **consistent**, especially if from **relaxed problems**



Wrap-up: A*



- A* uses both **backward costs** $g(n)$ and (estimates of) **forward costs** $h(n)$
- A* is **optimal** with **admissible (tree search) / consistent heuristics (graph search)**
- **Heuristic design is key**: often use **relaxed problems**!

Tree Search – Pseudo-code

```
function TREE-SEARCH(problem, fringe) return a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    for child-node in EXPAND(STATE[node], problem) do
      fringe ← INSERT(child-node, fringe)
    end
  end
end
```

Graph Search – Pseudo-code

```
function GRAPH-SEARCH(problem, fringe) return a solution, or failure
  closed ← an empty set
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      for child-node in EXPAND(STATE[node], problem) do
        fringe ← INSERT(child-node, fringe)
      end
    end
  end
```




What you just learned and what's next?

- **Informed Search:**
 - Heuristics
 - Greedy Search
 - A* Search
 - Admissible Heuristics
 - Graph Search
 - Consistent Heuristics
- Next week – **Graphs & Networks**

Questions?

A.DiStefano@tees.ac.uk